Assessment 1:

Take any low quality image and perform following operations on to it:

Output 1.
- Extend the missing background.
- Upscale image to higher resolution.

Output 2.
- Add an object.
- Remove an object.

You may use any open source model and AI technique to achieve this. Use of ComfyUI is recommended to generate the desired output.

Required Output Specifications:

- Minimum resolution: 720 pixels
- Orientation: Landscape
- Format: PNGs or high-quality JPEGs
- Attach a Comfy Workflow if assessment is done with ComfyUI.

Assessment 2:

Create a high-quality promotional key visual for a fictional consumer brand.
You are free to use any AI technique. Use of ComfyUI is recommended to generate the desired output.

Scenario:
A fictional premium headphone brand called **Auralis** is launching a new wireless headphone model named **Auralis Nova**.
Create a cinematic advertising image featuring the headphones as the primary subject.

Visual Direction:

- The final image should communicate:
- Premium quality
- Modern technology
- Immersive sound
- Sophisticated design
- Strong product focus

Required Output Specifications:

- Minimum resolution: 720 pixels
- Orientation: Landscape
- Format: PNG or high-quality JPEG
- The product must remain the dominant subject
- The composition must contain usable negative space for text
- The final image must not include accidental text, watermarks, signatures, or visible generation artifacts.

Assessment 3:

Create an image segmentation model that separates a target object from the image background.

Example use cases include:

- Separating a product from its background
- Segmenting roads in street images
- Detecting surface defects
- Segmenting pets
- Identifying plant leaves
- Separating people from the background

Use of ComfyUI for this assessment is preferable, but you may use any AI technique to generate the desired output.

Example Dataset Structure:

```
dataset/
├── images/
│   ├── image_001.jpg
│   ├── image_002.jpg
│   └── image_003.jpg
└── masks/
    ├── image_001.png
    ├── image_002.png
    └── image_003.png
```

Assessment 4:

Build a fully functional Kanban board with drag-and-drop task management. Tasks should move between columns (To Do, In Progress, Review, Done) and persist state locally.

Requirements:

- No external APIs or backend required
- Drag-and-drop functionality between columns
- Add, edit, and delete tasks with title, description, and priority (Low / Medium / High)
- Color-coded priority tags
- Task count per column
- LocalStorage persistence
- Responsive layout for desktop and tablet

Deliverables:

- Working Kanban board UI
- Smooth drag-and-drop interaction
- Task CRUD operations
- Clean, modern visual design

Sample Data:

```
[
{"id":1, "title":"Design homepage", "desc":"Create wireframes",
"priority":"High", "status":"todo"},
{"id":2, "title":"Setup CI/CD", "desc":"GitHub Actions pipeline",
"priority":"Medium", "status":"inprogress"},
{"id":3, "title":"Write API docs", "desc":"OpenAPI spec",
"priority":"Low", "status":"review"}
]
```

Evaluation Criteria:

- Smooth, bug-free drag-and-drop UX
- Visual polish and responsiveness
- Code organization and readability
- Prompt decomposition strategy

Bonus Challenge:
Add a search/filter bar and due-date support with overdue highlighting.

Assessment 5:

Build a functional Retrieval-Augmented Generation (RAG) application that allows a user to upload or select documents and ask questions based on their contents.

You may use AI coding assistants, documentation, open-source libraries, templates, and online resources. The assessment focuses on how effectively you use available tools to understand the problem, make technical decisions, build a working solution, and explain your approach.

Problem Statement:
Create a small RAG application that answers questions using information retrieved from a provided document collection.
The application should:

1. Load and process one or more documents.
2. Split the documents into meaningful chunks.
3. Generate and store embeddings.
4. Retrieve relevant chunks for a user question.
5. Send the retrieved context to a language model.
6. Display the generated answer.
7. Show the source text or document references used to generate the answer.

A sample dataset (PDF, text, or Markdown documents) should be included in the submitted repository.
